# 02

PERFORMANCE

# Performance Web

Carga inicial, change detection, renderizado de listas e imágenes.

*test2 · performance-optimization*

77.8 kB inicial | OnPush ×3 | lazy + async

PROYECTO
inventory-app · Angular 22

FECHA
Septiembre 2026

METODOLOGÍA
agent-skills · OWASP · WCAG 2.1

PENDIENTE lint | PENDIENTE build | PENDIENTE e2e

PENDIENTE evidencia

# Test 2 — Performance Web

**Fecha:** 2026-09-10 **Objetivo:** `C:\Users\juanca\Documents\app\inventory-app` (SPA Angular 22, backendless) **Skill aplicado:** `performance-optimization` + referencia `performance-checklist.md`
**Verificación:** `npm run lint` ✅ | `npm run build` ✅

---

## 1. Medición (baseline)

Sintético vía build de producción (`ng build`, defaultConfiguration production) + auditoría estática del frontend (no hay red/back-API, así que TTFB/CWV de campo no aplican).

| Métrica | Antes | Después | Notas |
|---|---|---|---|
| Initial total | 291.97 kB raw / **77.74 kB transfer** | 291.97 kB / **77.80 kB transfer** | Sin cambios materiales (±0.1 kB) |
| Lazy chunks (por ruta) | ~150 kB raw / ~38 kB transfer | igual | Code splitting por ruta ya activo |
| Budget inicial (angular.json) | warning 500 kB / error 1 MB | sin cambios | Actual 292 kB < 500 kB |

Target del skill: initial JS < 200 kB gzip → **cumplido** (77.8 kB estimated transfer ≈ gzip).

## 2. Hallazgos

| ID | Severidad | Hallazgo |
|---|---|---|
| P1 | MEDIA | `changeDetection` por defecto (no OnPush) en las 3 páginas más pesadas: **dashboard** (SVG charts + donut + gauges), **products** (tabla + filtros computed) y **reports** (agregados por categoría). Con el resto del árbol en OnPush, cada evento de zona re-renderizaba sus subárboles sin necesidad. |
| P2 | BAJA | `barColor()` invocada como **método en plantilla** (dashboard): re-ejecutaba la llamada por ítem en cada chequeo de CD. `catColors` ya era un mapa estable en el componente. |
| P3 | BAJA | Miniaturas de imágenes en listas (products/movements/suppliers/reports) sin `loading="lazy"` ni `decoding="async"`: decodificaban al pintar todo el listado aunque estuvieran fuera del viewport. (Contenedor `.thumb` tiene tamaño fijo 40×40 → sin riesgo de CLS.) |
| P4 | INFO | `dailySeries()` / `donut()` / `gauges()` se leen varias veces por plantilla → safe, ya son `computed` memoizados (accessor barato, cero recálculo). |
| P5 | INFO | Listas sin virtualización (products/movements). A escala del prototipo OK; **no se implementa** para no sobre-ingenierizar sin evidencia de problema (regla del skill: medir antes de optimizar) |

| ID | Severidad | Hallazgo |
|---|---|---|
| P6 | INFO | `styles.scss` ya incluye `prefers-reduced-motion` (línea 503) → animaciones desactivadas para usuarios con reduced-motion. |

## 3. Fixes aplicados

| ID | Archivo(s) | Cambio |
|---|---|---|
| P1 | `pages/dashboard/dashboard.component.ts`, `pages/products/products.component.ts`, `pages/reports/reports.component.ts` | Añadido `changeDetection: ChangeDetectionStrategy.OnPush`. Todas las lecturas de plantilla son signals/computed, así que la detección se dispara solo cuando cambian las dependencias. |
| P2 | `pages/dashboard/dashboard.component.ts` + `.html` | Método `barColor()` eliminado; uso directo de `catColors[c.cat]` (mapa estable). |
| P3 | `products.component.html`, `movements.component.html`, `suppliers.component.html`, `reports.component.html` | `loading="lazy" decoding="async"` en miniaturas de listas. |

No se tocó: budgets de `angular.json` (292 kB dentro de 500 kB), fuentes (sistema, sin @font-face externo), ni terceros (ninguno: 0 librerías JS externas).

## 4. Ledger de intentos (per skill)

| Idea | Baseline → Resultado | Veredicto | Por qué |
|---|---|---|---|
| OnPush en dashboard/products/reports | (bundle neutral) | **kept** | Elimina re-renderizado por zona en las páginas pesadas; trivial y reversible |
| `barColor()` → lookup `catColors` | (neutral) | **kept** | Asesina llamadas a método por ítem en el bucle; coste cero |
| `loading="lazy"` + `decoding="async"` en thumbs | (neutral, menos trabajo main-thread al pintar listas) | **kept** | Práctica estándar de checklist; sin efecto en bundle |
| Virtualización de listas | n/a | **no intentado** | Sin evidencia de problema a la escala actual; añadiría complejidad |
| Reducción adicional del bundle | 77.8 kB ya bajo target | **no intentado** | Ya < 200 kB gzip |

## 5. Verificación

- `npm run lint` → All files pass linting.
- `npm run build` → completado en ~3.5s; initial 77.80 kB transfer (linterna: `evidence/build-before.txt` vs `evidence/build-after.txt`).
- Resultado re-medido con el mismo comando (ng build) que la baseline.

## 6. Cierre

Sin regresiones. Bundle inicial **77.8 kB transfer** (cumple objetivo < 200 kB gzip). Los cálculos pesados del dashboard siguen memoizados (computed). Riesgo residual: mínimo; queda documentada la mejora futura de virtualización para escalar a miles de SKUs.